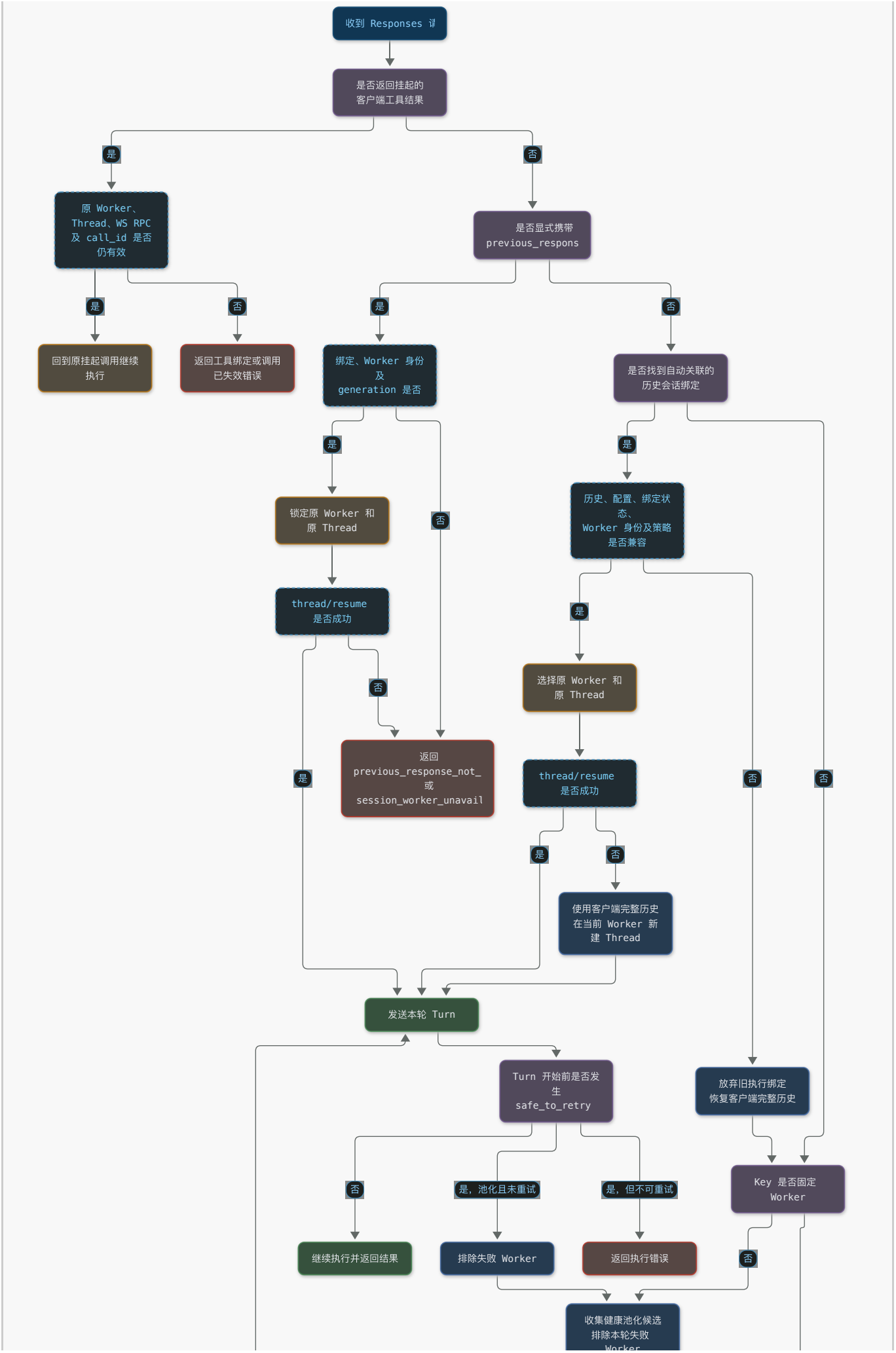
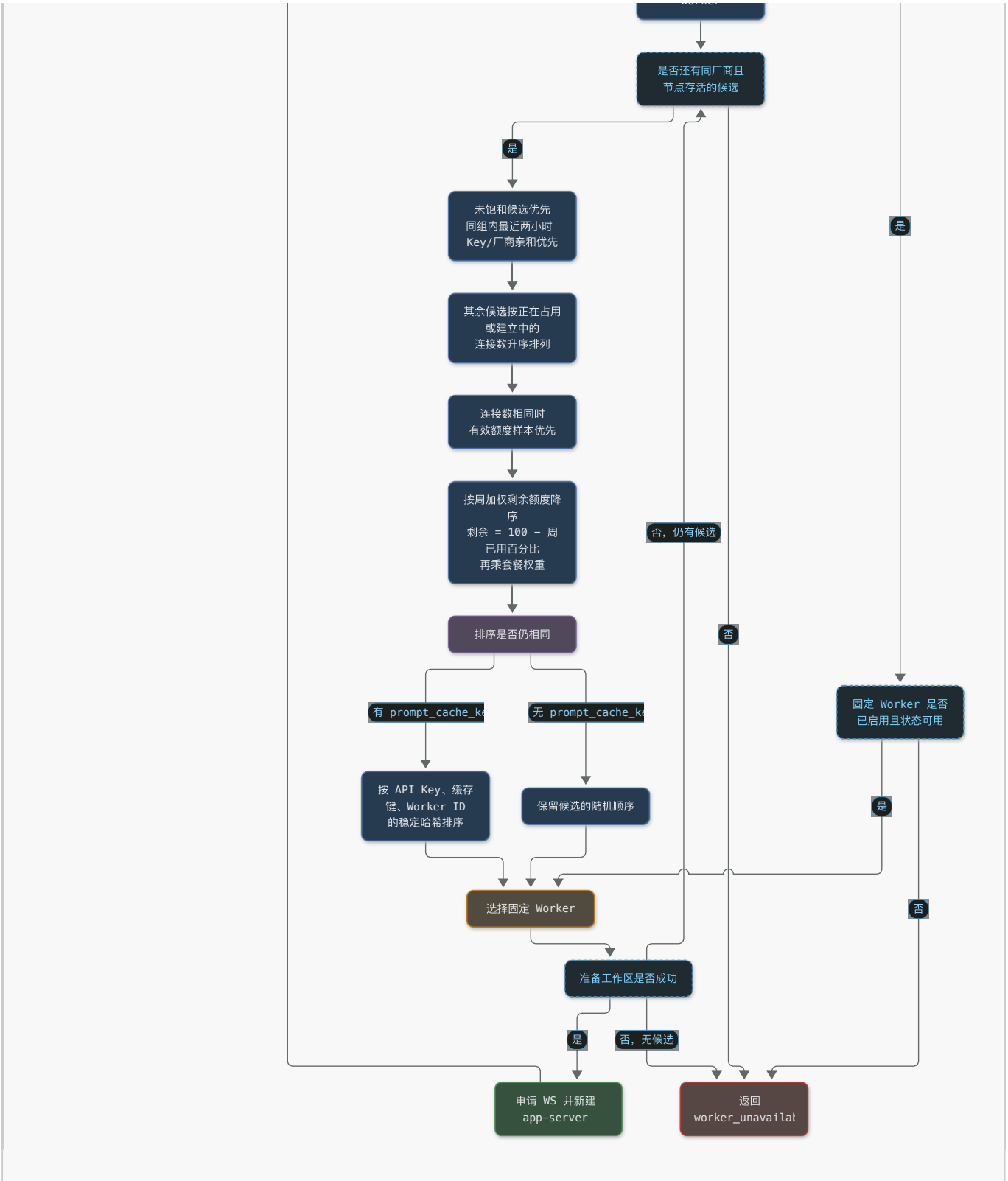
网关会话路由逻辑

当前路由分成两层：先判断能否继续原会话，再决定由哪个 Worker 执行。`/v1/responses` 和 `/v1/chat/completions` 共用执行准备与 Worker 选择逻辑，后者先转换为内部 Responses 请求。核心代码位于 main.py、execution.py、app_server.py、backend.py 和 cluster.py。

Worker 必须属于请求模型对应的厂商；模型可用性、Key 权限和参数能力校验先于调度。下文的 WS、`thread/start`、`thread/resume` 描述针对 Codex；Claude/Gemini 共用候选选择规则，但会话恢复由各自后端实现，不能直接套用 Codex 的重建规则。

总体流程图


收到 Responses 请
是否返回挂起的客户端工具结果
是
否
原 Worker、Thread、WS RPC 及 call_id 是否仍有效
是否显式携带 previous_respons
是
否
是
否
回到原挂起调用继续执行
返回工具绑定或调用已失效错误
绑定、Worker 身份及 generation 是否
是否找到自动关联的历史会话绑定
是
否
是
否
锁定原 Worker 和原 Thread
历史、配置、绑定状态、Worker 身份及策略是否兼容
thread/resume 是否成功
是
否
选择原 Worker 和原 Thread
返回 previous_response_not_ 或 session_worker_unavail
是
否
否
thread/resume 是否成功
是
否
使用客户端完整历史在当前 Worker 新建 Thread
发送本轮 Turn
Turn 开始前是否发生 safe_to_retry
放弃旧执行绑定恢复客户端完整历史
否
是，池化且未重试
是，但不可重试
Key 是否固定 Worker
继续执行并返回结果
排除失败 Worker
返回执行错误
否
收集健康池化候选排除本轮失败 Worker



图中紫色节点表示路由判断，蓝色虚线节点表示绑定或运行状态校验，橙色节点表示沿原绑定或固定策略选择 Worker，蓝色节点表示重新调度，绿色节点表示开始或继续执行，红色节点表示错误出口。

图中“周加权剩余额度”的计算公式为：

```
（100 − 周已用百分比）× 套餐权重
```

例如，已用 2%、权重 1 的 Worker 得分为 98；已用 15%、权重 10 的 Worker 得分为 850。在容量分组、近期 Key 亲和和活跃连接数均相同时，后者优先。

1. 全新对话

固定 Worker 的 Key

如果 Key 配置为 `pinned`，请求只会路由到该 Key 指定的 Worker。

该 Worker 必须同时满足：

- `enabled = true`
- 状态为 `ready` 或 `busy`
- 没有被删除
- 厂商与请求模型一致；启用集群时，所属节点必须存活

如果不满足，直接返回 `worker_unavailable`，不会自动换到池内其他 Worker。

自动池化的 Key

系统从所有满足以下条件的 Worker 中选择：

- 已启用
- 状态为 `ready` 或 `busy`
- 不在本次排除列表内
- 厂商与请求模型一致
- 启用集群时，所属节点存活

具体选择方式：

请求情况	Worker 选择方式
存在未饱和候选	未饱和候选整体排在饱和候选前，优先级高于近期 Key 亲和
池化请求有近期使用记录	在同一容量分组内，优先复用同一 Key、同一厂商过去两小时内最后使用的 Worker（计入 2xx 且无 error_code 的用量记录，以及 499 中断记录；记录必须有 worker_id）
无近期记录，或最近使用的 Worker 不可用	回退到池排序，优先选择正在占用的 WS 数最少的 Worker
活跃连接数相同	优先选择周加权剩余额度 `（100 − 周已用百分比）× 套餐权重` 最大的 Worker
连接数和额度分数都相同，且有 `prompt_cache_key`	按 `API Key + prompt_cache_key + Worker ID` 的稳定哈希顺序选择
连接数和额度分数都相同，且没有 `prompt_cache_key`	保留数据库随机顺序
最近两小时没有有效周额度数据	在同连接数下排在有有效额度数据的 Worker 后，再按缓存亲和或随机顺序选择

请求情况	Worker 选择方式
首选 Worker 工作区准备失败	隔离该 Worker，继续尝试下一个候选
所有候选均失败	返回 `worker_unavailable`

表中连接数、额度和缓存键排序用于同一容量分组中、近期亲和优先之后的候选。只查询最新一条符合条件的用量记录；该 Worker 不可用时，回退到池排序，不逐条追溯更早使用过的 Worker。

“活跃连接”指连接池中正在租用或建立中的 WS，空闲等待复用的持久 WS 不计入活跃数。容量饱和是另一个判断：对 Codex，如果当前 Key 没有可复用的空闲 WS，且该 Key/Worker 分组或整个 Worker 的连接槽位已达到上限，就标记为饱和。其他 Key 的空闲 WS 仍占用 Worker 槽位，因此活跃数较低也可能饱和。`busy` 状态仍属于候选状态，真正的并发上限由连接池控制。

饱和只影响候选顺序，不直接剔除 Worker。全部候选都饱和时，仍按近期亲和及池排序选择，再由后端执行容量准入或等待；不能把“饱和”直接等同于 `worker_unavailable`。固定 Key 和已有绑定不因容量排序而换 Worker。

集群模式下，网关汇总存活节点的实时负载和针对当前 Key 的饱和状态；远端负载查询失败时回退到该节点最近心跳的连接数，实际准入仍以后端为准。

周用量和权重来自最近一次 v3 `subscription_usage` 快照。快照每小时采集一次，只有采集时间距当前不超过两小时才参与路由。某个 Worker 的额度读取失败时，其分数保持未知；在活跃连接数相同的候选中，有有效额度数据的 Worker 优先于未知数据的 Worker。

`prompt_cache_key` 只帮助相同前缀优先落到相同 Worker，提高缓存复用机会，不创建会话，也不会覆盖已有会话绑定。

Worker 选定以后，系统才在这个 Worker 内：

1. 优先复用该 Key 的空闲 WS；
2. 没有空闲 WS 时按上限创建新连接；
3. 为本次 WS 使用独立工作目录；
4. 创建新的 app-server Thread。

2. 已有且有效的自动会话绑定

客户端携带稳定会话标识和完整历史时，系统会查询 `ExecutionSession` 和 `ResponseBinding`。

只有以下条件全部满足才恢复原 Worker 和 Thread：

- 历史是已保存历史的追加；
- 模型、instructions、工具和输出配置一致；
- ResponseBinding 仍为 `active`；
- Worker generation 没有改变；
- Worker 已启用且状态为 `ready` 或 `busy`；
- 固定 Worker 策略没有改变；
- 当前没有另一个请求持有该会话执行锁。

验证成功后：

```
逻辑会话
  → 原 ResponseBinding
  → 原 Worker
  → 该 Worker 上任意可用 WS
  → thread/resume 原 Thread
```

这里固定的是 Worker 和 Thread，不固定原来的 WS。

3. 原绑定 Worker 已失效

对于通过客户端会话标识自动找到的绑定，系统允许安全重建。

以下情况会放弃原绑定：

- Worker 被停用、删除或进入异常状态；
- Worker 更换了登录账号，generation 变化；
- Key 后来改成固定到另一个 Worker；
- ResponseBinding 被管理员释放；
- 原 Worker 在实际调度前变得不可用。

处理过程是：

```
原绑定无法使用
  → 清除自动恢复参数
  → 恢复客户端提交的完整历史
  → 按当前 Key 策略重新选择 Worker
  → 在新 Worker 上创建新 Thread
  → 发送本轮请求
```

如果 Key 是池化模式，就重新进入“未饱和优先 → 最近两小时 Key/厂商亲和 → 活跃连接数 → 有效额度样本 → 周加权剩余额度 → 缓存亲和/随机”的优选调度；如果 Key 是固定模式，就路由到当前指定的 Worker。

需要区分校验阶段和调度阶段：上述绑定失效可在 `prepare_execution` 中触发重建；若自动绑定通过校验后，原 Worker 在实际选择或工作区准备时才不可用，`choose_execution_target` 仅对池化 Codex 自动恢复请求再次选路，固定 Key 和其他厂商直接返回错误。

因此，自动关联的会话在原 Worker 失效时，可以转移到另一个 Worker，但会创建新的后端 Thread。上下文来自客户端提交的完整历史，而不是把原 Thread 跨账号迁移。

4. Worker 有效，但原 Thread 不存在

系统会先在原 Worker 上执行 `thread/resume`。

如果 `thread/resume` 返回 `AppServerError`（包括 Thread 不存在），并且这次恢复属于网关自动识别的 Codex 会话：

- 将旧绑定标记为不可恢复；
- 清除增量输入；

- 恢复客户端的完整历史；
- 仍在当前已选 Worker 上执行 `thread/start`；
- 创建新 Thread 后发送本轮请求。

这个场景通常不会重新选择 Worker，因为 Worker 本身仍然正常，只是 Thread 丢失。

5. 显式 `previous_response_id`

显式 `previous_response_id` 比自动会话关联更严格。

如果它指向的绑定：

- 已被释放；
- Worker 已删除；
- Worker 换过账号；
- generation 不一致；

系统返回 `previous_response_not_found`，不会静默换 Worker。

如果绑定仍有效，但 Worker 当前不是 `ready/busy`，返回 `session_worker_unavailable`。

如果 Worker 正常但 `thread/resume` 失败，也返回 `previous_response_not_found`，不会用完整历史自动重建。原因是显式请求有可能只携带本轮增量，网关无法证明自己拥有完整上下文。

6. 等待客户端工具结果的会话

工具调用必须回到原来的：

- Worker
- Thread
- WS 中挂起的 app-server RPC
- `call_id`

这类请求不能切换 Worker。Worker 被删除、换账号或挂起调用丢失时，会返回 `tool_worker_invalidated`、`tool_binding_invalidated` 或 `client_tool_call_unavailable` 等错误。

客户端只能发起一个携带完整历史的新用户轮次，让系统放弃旧工具调用并新建 Thread。

7. 执行前 Worker 突然失败

对于全新会话或自动恢复会话，如果满足以下条件：

- Key 是池化模式；
- 不是工具结果续接；
- Turn 尚未开始；
- 错误被标记为 `safe_to_retry`；

系统会排除刚刚失败的 Worker，再从池中选择另一个 Worker 重试一次。

第二次选择排除刚刚失败的 Worker，保持原厂商，并重新应用“未饱和优先 → 最近两小时 Key/厂商亲和 → 活跃连接数 → 有效额度样本 → 周加权剩余额度 → 随机顺序”，只重试一次。安全重试不会继续携带首次请求的

`prompt_cache_key` 亲和条件。自动恢复会话的跨 Worker 安全重试仅适用于 Codex；显式绑定和固定 Key 不参与此重试。

调度后仍可能发生容量竞争。池化新请求在存在其他未饱和候选时可设置 `allow_capacity_fallback`；后端只有返回 `safe_to_retry` 的容量错误才允许换 Worker。不可重试的容量错误通常返回 `worker_capacity_exceeded`，不会因此隔离 Worker。流式响应还要求尚未向客户端输出模型内容。

一旦 `turn/start` 可能已经执行，系统不再换 Worker，避免重复执行。

8. 路由优先级

整体优先级可以概括为：

```
挂起工具调用的原 Worker
        ↓
显式 previous_response_id 的原 Worker
        ↓
历史和配置验证通过的自动绑定 Worker
        ↓
Key 当前固定的 Worker
        ↓
同厂商、健康且节点存活的池化候选：未饱和优先
        ↓
同容量分组内，最近两小时同 Key/厂商的最后使用 Worker
        ↓
活跃连接数最少的 Worker
        ↓
有有效额度数据的 Worker
        ↓
周加权剩余额度最大的 Worker
        ↓
prompt_cache_key 亲和或随机兜底
```

前三项优先沿已有执行绑定续接；固定策略只使用指定 Worker，之后的排序用于池化新 Thread。近期 Key 亲和是软偏好，会让位于容量可用性；`prompt_cache_key` 只用于最终平局排序。**Codex 自动关联会话可在原 Worker 失效后，用完整历史在其他 Worker 重建；显式 `previous_response_id` 和仍挂起的工具 RPC 不会静默迁移。**